

Administración de procesos

1. Conceptos Fundamentales de Procesos

Un proceso es simplemente un programa en ejecución. En Linux, todo proceso tiene un identificador único llamado **PID** (Process ID) y toda la información de los mismos se almacena dinámicamente en el directorio virtual **/proc**.

Existe una jerarquía estricta: todo proceso (hijo) es iniciado por otro proceso (padre).

2. Los 3 Tipos de Procesos

Entender esta clasificación es clave para saber qué buscar:

- ~ Normales (Interactivos): Se ejecutan vinculados a una terminal y bajo el nombre de un usuario. Si cierras la terminal, el proceso muere.

- ~ Daemons (Servicios): Corren en segundo plano (background) sin salida directa a una terminal. Generalmente se inician con el sistema y se quedan "escuchando" peticiones a través de un puerto de red (ej. un servidor web Apache o el servicio SSH).

- ~ Zombies: Son procesos que ya terminaron su ejecución, pero siguen ocupando un lugar en la tabla de procesos porque su proceso "padre" no ha leído su señal de finalización. Generalmente indican un error de programación en el software.

3. Control de Trabajos (Foreground vs. Background)

Cuando ejecutas un comando normal, la terminal queda bloqueada hasta que termine (primer plano o *foreground*). Para evitar esto y seguir trabajando:

- ~ El operador `&`: Si agregas un *ampersand* al final de un comando (ej. `script_respaldo.sh &`), el proceso se envía inmediatamente a segundo plano (*background*). La terminal te devuelve el PID y queda libre para seguir usándola.
- ~ `jobs`: Muestra la lista de procesos que enviaste a segundo plano en tu sesión actual de terminal.
- ~ `fg` (Foreground): Trae un proceso que estaba en segundo plano de vuelta al primer plano. Se puede usar con el número de trabajo (ej. `fg 1`).

4. Inmunidad con `nohup` (No Hang Up)

El problema: Incluso si envías un proceso a segundo plano con `&`, si cierras tu sesión SSH o la terminal, el sistema le enviará una señal de "cuelgue" (SIGHUP) a todos tus procesos hijos y los matará. La solución: El comando `nohup`.

- ~ Sintaxis: `nohup [comando] &`
- ~ Uso: Inicia un proceso haciéndolo inmune al cierre de la terminal. Es obligatorio cuando te conectas por SSH a un servidor para lanzar un script largo (como un backup o un escaneo de red) y necesitas desconectarte sin que el trabajo se cancele.

Perspectiva de Ciberseguridad

Monitorizar procesos es la principal técnica de caza de amenazas (*Threat Hunting*). Un atacante que logre acceder a tu servidor intentará ocultar su malware. A menudo lo hacen ejecutando procesos *Daemons* con nombres engañosos que imitan servicios legítimos del sistema operativo. Además, una proliferación intencional de procesos *Zombie* puede ser un método para realizar un ataque de Denegación de Servicio (DoS), agotando los PIDs disponibles del sistema para que no pueda abrir ningún programa más.

Pongamos esto en un caso práctico

Imagina que te conectas por SSH a un firewall, lanzas un escaneo de puertos usando `nmap escaneo.sh &` (enviándolo a segundo plano) y a los 5 minutos se te corta el internet.

1. Primero ingresamos por ssh

```
> ssh luck4@192.168.122.162
The authenticity of host '192.168.122.162 (192.168.122.162)' can't be established.
ED25519 key fingerprint is SHA256:SQEPaJwD37TfI/hM1MA1CSuKNivApZM0CGqIBG9fyeY.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.122.162' (ED25519) to the list of known hosts.
luck4@192.168.122.162's password:
Linux UTN-Lucka 6.12.90+deb13.1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.90-2 (2026-05-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri May 29 10:05:26 2026 from 192.168.122.1
luck4@UTN-Lucka: $
```

2. Después lanzamos un escaneo con nmap (usamos localhost para no escanear redes externas sin permiso)

`nmap -p- -T2 localhost > escaneo.txt 2>&1 &`

(-p- escanea los 65535 puertos, -T2 hace que sea muy lento para darnos tiempo, > escaneo.txt guarda el resultado para que no ensucie tu pantalla, y el & final lo envía a segundo plano).

```
luck4@UTN-Lucka: $
luck4@UTN-Lucka: $ nmap -p- -T2 localhost > escaneo.txt 2>&1 &
[1] 1029
luck4@UTN-Lucka: $ jobs
[1]+  Running                  nmap -p- -T2 localhost > escaneo.txt 2>&1 &
luck4@UTN-Lucka: $ ps aux | grep nmap
luck4      1029  0.0  0.3 29560 13628 pts/0    S   08:46   0:00 nmap -p- -T2 localhost
luck4      1032  0.0  0.0  6520  2292 pts/0    S+  08:46   0:00 grep nmap
luck4@UTN-Lucka: $
```

Con `jobs` lo veremos que está corriendo.

Y con `ps aux | grep nmap` veremos su PID real.

3. Para simular el corte de conexión simplemente cerramos la terminal conectada por ssh, luego nos volvemos a conectar.

```
luck4@UTN-Lucka:~$ ps aux | grep nmap
luck4      1026  0.0  0.0   6520  2320 pts/0    S+   13:01   0:00 grep nmap
luck4@UTN-Lucka:~$
```

Poner un **&** al final de un comando solo lo manda a segundo plano para liberarte la consola, pero el proceso sigue siendo "hijo" de esa ventana SSH. Al cerrar la ventana "X", la sesión SSH le envió la señal de muerte (**SIGHUP**) a todos sus hijos, matando el escaneo a la mitad.

4. Esto se soluciona usando el comando `nohup`, ante cualquier corte de red, el proceso seguirá corriendo sin cortarse.

```
luck4@UTN-Lucka:~$ nohup nmap -p- -T2 localhost > escaneo.txt 2>&1 &
[1] 1101
```

Por último un resumen para tener a mano.

